GEOMETRIC AND PERFORMANCE ANALYSIS OF RECURSIVE MULTI-AGENT SYSTEMS FOR ENTERPRISE PROCESS AUTOMATION

A Thesis

Submitted in partial fulfilment for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE)

by

NAVEEN SANKAR R S (CB.SC.U4AIE23323)
RISHI RAAJHA A (CB.SC.U4AIE23335)
SAKTHI PRANAV S (CB.SC.U4AIE23336)
HARSHVARDHAN V (CB.SC.U4AIE23353)



DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE CAMPUS (INDIA)

28 October 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE CAMPUS



BONAFIDE CERTIFICATE

This is to certify that the thesis entitled "**Geometric and Performance Analysis of Recursive Multi-Agent Systems for Enterprise Process Automation**" submitted by **Naveen Sankar R S (CB.SC.U4AIE23323)**, **Rishi Raajha A (CB.SC.U4AIE23335)**, **Sakthi Pranav S (CB.SC.U4AIE23336)** and **Harshvardhan V (CB.SC.U4AIE23353)** for the award of the **Degree of Bachelor of Technology** under the **Faculty of Engineering** is a bonafide record of the work carried out by them under my guidance and supervision at "Department of Artificial Intelligence", Amrita Vishwa Vidyapeetham, Coimbatore Campus.

Submitted for the examination held on 28 October 2026

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Dr. Prem Jagadeesan
Thesis Advisor,
Assistant Professor,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE
AMRITA VISHWA VIDYAPEETHAM
COIMBATORE CAMPUS

DECLARATION

We, **Naveen Sankar R S** (CB.SC.U4AIE23323), **Rishi Raajha A** (CB.SC.U4AIE23335), **Sakthi Pranav S** (CB.SC.U4AIE23336) and **Harshvardhan V** (CB.SC.U4AIE23353) hereby declare that this thesis entitled "**Geometric and Performance Analysis of Recursive Multi-Agent Systems for Enterprise Process Automation**" is the record of the original work done by us under the guidance of **Dr. Prem Jagadeesan**, Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore Campus. To the best of our knowledge, this work has not formed the basis for the award of any degree/diploma/associateship/fellowship or a similar award to any candidate in any University.

Place: Coimbatore **Signature of the Students**
Date: 28-10-2026

COUNTERSIGNED

Dr. K. P. Soman
Professor and Dean,
Department of Artificial Intelligence,
Amrita Vishwa Vidyapeetham,
Coimbatore Campus,
India.

Contents

Acknowledgements **iii**

List of Figures **iv**

List of Tables **v**

Abbreviations **vi**

List of Symbols **vii**

Abstract **viii**

1 Introduction **1**

1.1 Background and Motivation 1
1.2 Problem Statement 1
1.3 Project Objectives 2
1.4 Summary of Contributions 2
1.5 Thesis Organization 2

2 Literature Review **4**

2.1 Foundational Reasoning and Verbal Reflection Frameworks 4
2.2 Multi-Agent Collaboration and Communication Bottlenecks 4
2.3 Benchmarking Tool-Agent Interactions in Stateful Enterprise Domains 5
2.4 Recursive Multi-Agent Systems and Latent Deliberation 5
2.5 Neural Representational Geometry in Language Models 6
2.6 Synthesis, Delineation of Contributions, and Identified Research Gap 6

3 System Architecture and Experimental Methodology **7**

3.1 Two-Tier System Topology and Agent Functional Roles 7
3.2 Foundation Model Backbone, Local Serving Infrastructure, and Reproducibility 7
3.3 Neural Recursive Adapters, Pretraining Protocol, and Tensor Routing 9
3.4 Enterprise Benchmark Environment: τ^2-Bench Retail Domain 10
3.5 Operational Evaluation Metrics and Rubric 11
3.6 Composite Task Complexity Metric and Statistical Testing Methodology 12

4 Geometric and Error Analysis of Latent Representations **14**
4.1 Theoretical Formulation of Geometric Representation Metrics 14
4.2 Inferences from Representation Drift and Convergence Dynamics 15
4.3 Dimensionality Analysis and Subspace Compression 16
4.4 Failure Diagnostics and the Null Outcome Relationship 17

5 Results and Discussions **18**
5.1 Quantitative Benchmark Performance Evaluation 18
5.2 Behavioral Accuracy and Computational Scaling 19
5.3 Geometric State Convergence and Dimensionality Results 21
5.4 Discussion of the Tool Selection Trade-Off 22

6 Conclusion and Scope for Further Research **23**

A Benchmark Environment and Tool Schema Specification **24**
A.1 Retail Domain API Tool Specifications 24
A.2 Relational Database Schema 25

References **26**

Acknowledgements

We express our deep gratitude and sincere thanks to our respected project supervisor, Dr. Prem Jagadeesan, Assistant Professor, Department of Artificial Intelligence, for his constant technical guidance, valuable insights, and constructive critique throughout the execution of this thesis.

We extend our sincere thanks to Dr. K. P. Soman, Professor and Dean, Department of Artificial Intelligence, for providing excellent laboratory infrastructure, GPU computational resources, and academic leadership.

We also thank the faculty and staff members of the Department of Artificial Intelligence for their support and academic cooperation. Finally, we thank our parents and peers whose encouragement enabled the completion of this research.

NAVEEN SANKAR R S
RISHI RAAJHA A
SAKTHI PRANAV S
HARSHVARDHAN V

List of Figures

3.1 System architecture. Multi-Round Text ReAct with verbal critique loops vs Recursive Latent ReAct passing continuous tensors through neural adapters in a two-tier design. 8

5.1 Total task completion accuracy as a function of recursive depth ($r = 0, 1, 2, 3$) for 114 corporate jobs comparing Multi-Round Text ReAct to Recursive Latent ReAct. 20

5.2 Table 5.1 shows the average computational inference expenditure per task run (TFLOPs) over recursion depths ($r \in \{0, 1, 2, 3\}$), comparing single-run scaling curves with benchmark-wide cumulative totals. 20

5.3 95% bootstrap confidence intervals for stepwise representation drift over recursive passes. 21

5.4 Over 200 bootstrap subsets were used to evaluate the participation ratio and effective dimensionality across recursive passes. 21

5.5 A null connection with global representation geometry was confirmed by comparing clean successes against failures using effect sizes (Cliff's δ with 95% bootstrap confidence intervals) across five possible trajectory features. 22

List of Tables

5.1 Overall Benchmark Performance Summary for 912 Execution Runs on τ^2-Bench Retail Domain. 19
5.2 Benchmark Evaluation Breakdown by Recursion Depth ($r = 0, 1, 2, 3$), with a mean continuous accuracy of goal satisfaction ($S_{\text{goal}} \times 100\%$). 19
5.3 Task Complexity Tier Performance and Execution delay at Terminal Depth $r = 3$ (single-run execution delay and reported continuous goal fulfillment accuracy $S_{\text{goal}} \times 100\%$). 19

A.1 Executable application programming interface tool specs. 24

Abbreviations

ReAct	–	Reason + Act Paradigm
MAS	–	Multi-Agent System
RecursiveMAS	–	Recursive Multi-Agent System
LLM	–	Large Language Model
PCA	–	Principal Component Analysis
ER	–	Effective Rank
GGUF	–	GPT-Generated Unified Format
API	–	Application Programming Interface
JSON	–	JavaScript Object Notation
VRAM	–	Video Random Access Memory
FLOPs	–	Floating Point Operations
SVD	–	Singular Value Decomposition

List of Symbols

h	–	Hidden state representation vector ($h \in \mathbb{R}^{d_h}$)
d_h	–	Hidden dimension size ($d_h = 2560$)
r	–	Recursion round / depth index ($r \in \{0, 1, 2, 3\}$)
R_{in}	–	InnerLink intra-agent residual adapter
R_{out}	–	OuterLink inter-agent projection adapter
D_r	–	Step-wise representation drift from round $r-1$ to r ($r \in \{1, 2, 3\}$)
D_{ij}	–	Inter-agent Euclidean representation distance
ER	–	Effective rank of representation manifold
θ_k	–	Principal subspace angle between success and failure manifolds
Δ_{SF}	–	Centroid separability between success and failure clusters
C	–	Composite task complexity metric

Abstract

Enterprise process automation requires autonomous reasoning agents that can manage stateful transactions, make deterministic API calls and adhere to organizational policy. Reason-and-Act frameworks separate reasoning traces from environment actions. But in multi-step processes, verbal critique loops pile up prompt text, rack up token costs, and allow errors to accumulate. We present in this thesis the design, geometric characterization and empirical benchmarking of a Recursive Multi-Agent System on continuous latent representations. Only at the boundary of database execution the system decodes to text, passing continuous hidden tensors between recursive rounds. The deliberation is shared among specialized Planner, Tool-Caller and Verifier agents connected via neural residual adapters.Latent ReAct achieves a mean task goal accuracy across recursion depths of 50.9 percent compared to 50.3 percent for Text ReAct (+0.6 pp) on the stateful τ^2-Bench Retail domain, evaluating 912 execution trajectories across balanced complexity tiers. This increases to 56.4 percent versus 54.4 percent at terminal depth (+2.0 pp). Latent ReAct achieves 50.8% accuracy compared to 45.9% for Text ReAct on the challenging Hard workflow tier, which shows the most improvements in goal accuracy. Clean terminal successes rise from 30 to 33 across the entire benchmark of 114 jobs, indicating a 10% relative increase. In addition to an observed descriptive deficit of 3.3 percentage points in single-step tool selection accuracy (bootstrap 95% CI: $[-5.1\%, -1.5\%]$), Latent ReAct simultaneously achieves a 26.1% reduction in latency, a 25.2% saving in generated tokens, and a 23.1% decrease in enterprise policy violations.

Geometric analysis of the underlying representation manifolds reveals that Latent ReAct exhibits a seventy-two percent step-wise drift contraction across the whole benchmark cohort ($p = 1.9 \times 10^{-20}$, maintaining seventy percent contraction with $p = 7.1 \times 10^{-15}$ under duplicate-filtered robustness testing). Trajectory stabilization into confined solution zones while preserving effective dimensionality without collapse ($18.9 \rightarrow 15.2$ dimensions) is empirically demonstrated by this. Internal agent states were analyzed and revealed near-orthogonal functional specialization in Latent ReAct ($\text{CosSim} \leq 0.087, D_{ij} \approx 51.5$). In contrast, Text ReAct is highly overlapped in words ($\text{CosSim} > 0.998$). The statistical analysis does not give a significant result and I report it as such. Task performance is not linearly separable from the states averaged globally. Further exploration of tool-specific localized representations might lead to a better prediction of execution reliability.

Chapter 1

Introduction

1.1 Background and Motivation

Customer service, logistics fulfillment, warranty processing and inventory management are increasingly relying on autonomous decision-making systems.

These agents are more than just question answerers. They must call external APIs, look up company policies, decompose complex customer goals, verify relational database status, process transaction updates and make sure all final constraints are met.

Instead of producing an answer in one feed-forward pass, an agent would work iteratively. It is thinking. It chooses a tool. It sucks up what comes back.

The wrong tool, badly formatted parameters or a rushed assumption can snowball into policy violations or outright task failure. Modern multiagent systems avoid these errors by assigning specialized roles and using recursive reflection. The catch is conventional systems can only speak text. Agents report their intentions, execution logs, critiques and error messages to us as natural language strings.

To overcome these limitations, this work proposes multi-agent recursive architectures communicating directly in continuous latent space. Continuous hidden tensors can flow between the specialized agent roles through parameterized neural adapters. This allows deliberation to occur in the high dimensional internal geometry of the model itself.

1.2 Problem Statement

In complex enterprise workflows of multi-agent language systems, text-based reflection results in heavy token overhead, high inference delay and accumulated errors as tasks get harder. There is no literature on continuous latent-state communication in stateful, tool-driven environments, empirically or geometrically. This thesis describes the design, implementation, and testing of a Recursive Latent Multi-Agent System against traditional Multi-Round Text ReAct in an enterprise customer service domain. It asks whether recursive latent communication can enable more robust and less computationally expensive task completion. Also, it asks about stability of trajectories in bounded solution regions with no dimensional collapse.

1.3 Project Objectives

The work consists of four parts . In the first step, I stratified 114 enterprise retail tasks from τ2-Bench into balanced Easy, Medium and Hard strata, resulting in 912 trajectories . Third, I developed geometric manifold metrics to mathematically characterize trajectory stabilization. Third, we evaluated runtime failure diagnosis with predictive linear probing. Fourth, I built a modular two-tier multiagent architecture with dedicated Planner, Tool-Caller, and Verifier agents, all running on a single frozen Qwen3.5-4B foundation model and connected via neural residual adapters.

1.4 Summary of Contributions

An empirical benchmark evaluation and geometric manifold characterisation of continuous latent deliberation in recursive multi-agent systems are presented in this thesis. The principal contributions are: First, we show that Latent ReAct offers significant operational benefits on intricate multi-step workflows, increasing Hard-tier goal accuracy by almost five percentage points (50.8% versus 45.9%) while executing 26.1% faster, producing 25.2% fewer tokens, and lowering enterprise policy violations by 23.1%. The mean goal fulfillment accuracy increases by 0.6 percentage points throughout recursion depths (50.9% against 50.3%) for the entire benchmark and by 2.0 percentage points at terminal depth (56.4% against 54.4%), in addition to a 3.3 percentage point descriptive disadvantage in the precision of single-step tool selection (bootstrap 95% CI: $[-5.1\%, -1.5\%]$). Second, we provide empirical evidence of representational trajectory stabilization: Latent ReAct stabilizes deliberation trajectories into localized solution regions without experiencing catastrophic dimensional collapse ($18.9 \rightarrow 15.2$ effective dimensions), achieving a 72% step-wise drift contraction across the entire benchmark cohort ($p = 1.9 \times 10^{-20}$, maintaining a 70% contraction with $p = 7.1 \times 10^{-15}$ under duplicate-filtered robustness testing). Third, verbal critique passing results in high context overlap (CosSim $>$ 0.998). We tackle this by continuous latent routing via neural adapters, keeping agent roles nearly orthogonal and functionally specialized (CosSim $<=$ 0.087, Dij 51.5). Finally, the failure analysis yields a null result after the multiple hypotheses correction. Global, role-averaged representations do not linearly predict downstream task success. So, for large systems, local, tool-specific population representations may be better predictors of execution reliability than geometry, and may warrant further study.

1.5 Thesis Organization

The remainder of the thesis is organized as follows. Chapter 2 surveys the literature on representation geometry, recursive self-reflection, multi-agent frameworks, and ReAct. Chapter 3 covers the system architecture, the neural adapters, and the experimental setup. Chapter 4 is on the geometric and error analysis of the latent representations. Chapter 5 deals with bench-

mark performance, complexity scaling and geometric visualizations. Main results, limitations and suggestions for future work are concluded in Chapter 6. Appendix A describes the design of relational database and the benchmark API tools.

Chapter 2

Literature Review

2.1 Foundational Reasoning and Verbal Reflection Frameworks

Interactive decision-making architectures have emerged as a more powerful alternative to static prompt generation in the development of autonomous reasoning systems powered by Large Language Models. ReAct integrates natural language reasoning with commands in external software. It showed substantial gains over standard chain-of-thought prompting on multi-hop question answering and decision-making benchmarks.

To reduce early execution errors, Shinn et al. [1] proposed Reflexion, an architecture for language agents to obtain verbal reinforcement learning through linguistic self-reflection. When a trajectory fails, the agent produces a textual evaluation of the trajectory and stores it in working memory for future rollouts. However, the nature of natural language constrains standard ReAct and verbal reflection frameworks. Suppose an agent gets a wrong return from a database or chooses a wrong API schema at the beginning. Later criticisms of it tend to be verbose, illogical, or obsessed with the earlier mistake. The context window fills with the critique text, one token at a time. And so you have a compounding failure loop that leaves the job un-done.

2.2 Multi-Agent Collaboration and Communication Bottlenecks

Single agents pipelines are unstable and the cognitive system for complex reasoning is heavy loaded, thus multi-agent systems are very popular. Wu et al. [2] proposed a modular framework (AutoGen) that enables different expert personas to conversate with each other. Agents like planners, programmers and code reviewers break down complex goals and coordinate with structured prompts. Specialization helps break down problems, but traditional configurations still communicate only in text. Agents must serialize plans, summary of status and critiques into natural language tokens that can be parsed and re-encoded by the next agent. This imposes severe operational limitations on stateful business tasks that need to collaborate iteratively,

very often. This dialog history increases the prompt context in each recursive cycle, resulting in a sharp increase in inference latency and computational cost. Unconstrained natural-language feedback is also mathematically unstable, so agents tend to oscillate between contradictory explanations, rather than converge on an answer.

2.3 Benchmarking Tool-Agent Interactions in Stateful Enterprise Domains

We need interactive simulators with programmable APIs, ordered states, and realistic organizational rules to evaluate autonomous agents in realistic enterprise settings. Yao et al. [4] proposed τ-Bench, a benchmark environment to evaluate the interactions between tool-agent-user in enterprise domains such as retail customer service and airline reservations. Against static question-answering datasets, τ-Bench offers an executable environment with relational databases, explicit company policies, and simulated user personas.

The benchmark reveals that even the best proprietary models are significantly less reliable in multi-step workflows . These workflows involve database changes, policy enforcement checks and long conversations.

This makes τ-Bench a challenging and well-suited testbed to explore the dependability of recursive agents.

2.4 Recursive Multi-Agent Systems and Latent Deliberation

In order to get around the bottleneck in textual communication, recent research has pioneered recursive deliberation directly within continuous latent representation spaces. In a groundbreaking study, Yang et al. introduced the Recursive Multi-Agent System (RecursiveMAS) framework [3]. RecursiveMAS reframes multi-agent cooperation as a single recursive computation loop using lightweight neural modules called RecursiveLink. Instead of decoding and forwarding natural language strings across intermediate deliberation rounds, agents pass continuous hidden-state tensors directly across representation spaces, decoding tokens strictly at the end task boundary. On mathematical problem solving and code generation benchmarks, Yang et al. demonstrated that latent recursion performs better than conventional text-mediated multi-agent pipelines in terms of correctness, inference latency, and significant token savings.

Zhu et al. [4] introduced LoopLM and the Ouroboros (Ouro) model family to supplement recursive agent systems in order to explore the scalability of latent reasoning using looping language models. By repeatedly processing internal activations through parameter-shared Transformer layers, LoopLM demonstrated that iterative computation in latent space is an effective substitute for verbose chain-of-thought generation, greatly lowering output token costs while enhancing reasoning depth. By demonstrating that continuous hidden representations capture

rich relational information over repeated runs, their findings offer strong architectural support for expanding latent looping into multi-agent tool execution contexts.

2.5 Neural Representational Geometry in Language Models

Task performance is measured by behavioral standards, but the internal representational mechanisms influencing model choices are not explained by them. To get at these internal representations, researchers have borrowed techniques from the geometry of high-dimensional neural data. Irie and Sompolinsky [7] characterized the representational geometry of concepts in large language models. They demonstrated that such networks organize semantic categories and associated concepts into structured low-dimensional manifolds. They also showed that cognitive changes in deep networks can be mathematically characterized as a trajectory flow in representation manifolds . Therefore, geometric measures give a direct insight into the internal dynamics of a network. Then Sorscher et al. [8] and Roy and Vetterli [9] formalized metrics like the Participation Ratio and effective rank quantifying the continuous dimensionality spanned by neural manifolds. They give us tools to measure subspace compression and to detect catastrophic dimensional collapse. Standard geometric formulas can be used to compute the principal angles between linear subspaces, as shown by Björck and Golub [10]. This provides a strict way to monitor manifold alignment and separability over agent roles and task phases.

2.6 Synthesis, Delineation of Contributions, and Identified Research Gap

The literature as a whole shows a clear lack of coherence. However, AutoGen [3] formalizes the agent specialization and τ-Bench [4] provides the stateful enterprise environment.

To my knowledge, based on the literature I surveyed, no one has looked at continuous latent recursive multi-agent deliberation in stateful enterprise tool-use systems controlled by relational databases and strict business policy constraints.

This thesis fills this gap. We evaluate a two-tier latent multi-agent architecture against Multi-Round Text ReAct on the stateful τ^2-Bench Retail domain over 912 execution rollouts. It also provides a full geometric description of the representation manifolds underlying it.

Chapter 3

System Architecture and Experimental Methodology

3.1 Two-Tier System Topology and Agent Functional Roles

I propose a two tier multi agent business system architecture which decomposes complex decision making into smaller chunks. It has three expert agents. The first tier is the Planner Agent, A1 . It figures out what the user really needs, checks operational constraints against company policy and builds a detailed execution plan.

The second level is execution and validation. The Planner continuously sends representations of plans to Tool-Caller Agent (A 2). A 2 maps each planned action to a particular API schema, resulting in structured JSON arguments. The Verifier Agent (A3) verifies the policy adherence and observes the external environment response. It also ensures that the changes in the database are in sync with the customer request and in case something goes wrong or does not add up, it sends back corrective feedback.

Figure 3.1 illustrates the overall system architecture, and compares the two operation approaches. First, the classic Multi-Round Text ReAct. The second is Recursive Latent ReAct, which routes continuous tensors among agents. For the text-based baseline, each handoff between agents maps to generating one natural language critique token at a time. This bloats the context and increases inference latency. Recursive Latent ReAct stores intermediate deliberation in continuous representation space, and only decodes tokens on the boundary of database execution.

3.2 Foundation Model Backbone, Local Serving Infrastructure, and Reproducibility

All comparisons are on the same Qwen3.5-4B model [11] ($P = 4.4 \times 10^9$ total parameters), so the results reflect differences in architecture design, not base capacity. This model includes

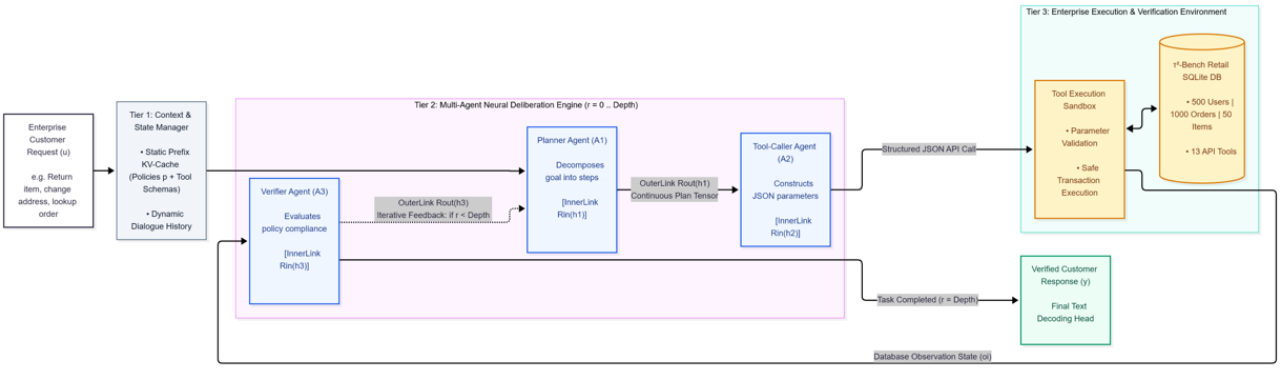


Figure 3.1: System architecture. Multi-Round Text ReAct with verbal critique loops vs Recursive Latent ReAct passing continuous tensors through neural adapters in a two-tier design.

embedding layers with a vocabulary size of 248,320, 32 hybrid Transformer layers interleaving Gated DeltaNet linear attention with gated full self-attention [5], a hidden dimension of $d_h = 2560$, 16 attention heads, and an 8,000-token active context budget. The model base weights were quantized to 4-bit medium block precision using GGUF (Q4_K_M), which produced a tiny 2.74 GB VRAM footprint. During forward passes, 4-bit quantized weights are dequantized in order to utilize 16-bit floating-point (bfloat16) activation vectors and 32-bit floating-point (float32) accumulation in CUDA cuBLAS GEMM kernels. Crucially, all neural adapters ($R_{\text{in}}, R_{\text{out}}$), activation vectors ($h \in \mathbb{R}^{2560}$), layer normalizations, and gradient updates must function in unquantized floating-point precision (bfloat16/float32) to prevent quantization noise in latent routing.

To ensure precise reproducibility, we provide the hardware and software configurations. For tool wrapper unit testing and local prototyping, an NVIDIA GeForce RTX 3050 laptop GPU (4 GB GDDR6 VRAM, Ampere architecture, 60W TGP) was utilized. The entire benchmark evaluation matrix (912 execution trajectories) and offline neural adapter pretraining were performed on a dedicated compute node with an NVIDIA RTX A4000 GPU (16 GB GDDR6 VRAM) running Ubuntu 22.04 LTS with CUDA 12.4, Python 3.10, PyTorch 2.4.0, and `llama-cpp-python` (v0.2.89 with FlashAttention-2 and cuBLAS). By completely offloading all 32 layers to GPU memory (`n_gpu_layers=33`), host paging and network variability were eliminated.

Fully deterministic decoding for all agents using fixed random seed 42 and greedy decoding (temperature 0.0, top-p 1.0). Token budgets are strictly restricted to 100 tokens for Tool-Caller schema outputs and 120 tokens for Verifier critiques. Monotonic hardware timers are used to measure wall-clock latency at the microsecond level.

3.3 Neural Recursive Adapters, Pretraining Protocol, and Tensor Routing

Our latent architecture controls communication between agent roles via continuous hidden representations rather than discrete textual tokens. $h \in \mathbb{R}^{d_h}$ represents the hidden state vector that was obtained from the final Transformer layer (Layer 32), with the hidden dimension $d_h = 2560$. During recursive passes, decision-making is mediated by two specialized neural adapter modules that employ full precision floating-point arithmetic.

The InnerLink adapter, denoted by R_{in}, does intra-agent representation refinement by updating an agent's internal latent state over recursive steps while preserving its functional specialization. It is built as a two-layer residual Multi-Layer Perceptron with pre-layer and post-layer normalization:

$$R_{\text{in}}(h) = \text{LayerNorm}_{\text{post}}\left(h + W_2\,\sigma\left(W_1\,\text{LayerNorm}_{\text{pre}}(h) + b_1\right) + b_2\right) \tag{3.1}$$

where $W_1 \in \mathbb{R}^{2560\times2560}$ and $W_2 \in \mathbb{R}^{2560\times2560}$ are projection weight matrices, and σ is the Gaussian Error Linear Unit (GELU) activation function. The residual skip connection ensures numerical stability over iterative recursive runs by preserving the original semantic representation.

The OuterLink adapter, denoted by R_{out}, manages cross-agent subspace projection. From the Planner to the Tool-Caller ($R_{\text{out}}^{\text{RT}}$), from the Tool-Caller to the Verifier ($R_{\text{out}}^{\text{TR}}$), and from the Verifier back to the Planner, it translates representations between various agent functional roles. Since different agent roles operate in distinct conceptual subspaces, R_{out} combines a straight linear residual projection with a higher-dimensional non-linear transformation:

$$R_{\text{out}}(h) = \text{LayerNorm}_{\text{tgt}}\left(W_{\text{res}}h + W_2\,\sigma\left(W_1\,\text{LayerNorm}_{\text{src}}(h) + b_1\right) + b_2\right) \tag{3.2}$$

where $W_1 \in \mathbb{R}^{5120\times2560}$, $W_2 \in \mathbb{R}^{2560\times5120}$, $W_{\text{res}} \in \mathbb{R}^{2560\times2560}$, and σ is GELU.

Adapter weight matrices were initialized via He normal initialization with zero-initialized bias vectors, and the direct residual projection matrix W_{res} was initialized as the identity operator. To establish role-specific and cross-agent coordination prior to benchmark deployment, the adapters were pretrained on an offline deliberation trajectory dataset comprising 10,000 multi-agent reasoning demonstrations split into an 80% training split (8,000 trajectories) and a 20% validation split (2,000 trajectories) with a fixed random seed of 42. These pretraining demonstrations were created by teachers enforcing sequential planning, tool invocation, and reflection trails throughout generic multi-step problem solving. Crucially, the pretraining data did not contain or overlap with any of the 114 τ^2-Bench retail customer support tasks, tool definitions, customer profiles, or target database states. This ensures that the pretraining corpus is kept separate from the evaluation environment to avoid benchmark contamination.

Using the AdamW optimizer ($\beta_1 = 0.9, \beta_2 = 0.95$, weight decay of zero), InnerLink training optimized role-specific latent feature extraction with an initial learning rate of 5×10^{-4} decayed

via a cosine schedule across twenty thousand steps with ten warmup steps and a batch size of two. The inner objective minimized a composite alignment loss:

$$\mathcal{L}_{\text{inner}} = 1.0 \times (1 - \text{CosSim}(h_{\text{pred}}, h_{\text{target}})) + 0.1 \times \|h_{\text{pred}} - h_{\text{target}}\|_2^2 \tag{3.3}$$

where h_{target} represents the role-specific activation vector extracted from particular planning or tool-calling demonstrator traces. OuterLink training jointly optimized cross-agent bridging across three recursive deliberation passes ($r = 3$) with frozen inner adapters under the supervision of terminal cross-entropy token generation loss paired with intermediate critique loss ($\lambda = 0.1$) using AdamW (learning rate 5×10^{-4}, gradient clipping norm of one point zero). Model checkpoints were evaluated every 1,000 steps on the held-out validation split. The checkpoint that had the lowest validation composite loss—checkpoint-10000 for InnerLink and the lowest joint validation cross-entropy plus critique loss for OuterLink—was selected. Throughout the benchmark evaluation, all adapter settings were executed precisely when frozen in evaluation mode. The Planner generates a hidden tensor $h_1^{(r)}$, which is refined by R_{in} and projected by $R_{\text{out}}^{\text{RT}}$ in order to condition the Tool-Caller without generating textual tokens. This entire process occurs in latent space. The Tool-Caller strictly decodes to structured JSON tokens in order to carry out the external API instruction. The execution observation is then sent to the Verifier via $R_{\text{out}}^{\text{TR}}$.

3.4 Enterprise Benchmark Environment: τ^2-Bench Retail Domain

We evaluate autonomous decision-making in a stateful enterprise environment using the Retail domain of the τ^2-Bench benchmark [6]. Thirteen deterministic Python Application Programming Interface tools and an interactive SQLite relational database are used in the benchmark to replicate a production-level customer service environment.

The database consists of 500 customer profiles and 1,000 orders with multiple items. It also has large product catalogs with stock levels, warehouse locations and return tracking records. The executable tools include user identification, order look up, pending order updates, item returns, product exchanges, refund calculations and promotional discount validation. The environment is operated to strict company procedures. Sorry, we do not accept returns after thirty days, we do not change orders once they have been delivered, and there are restocking fees on certain categories of items. A formal definition of an enterprise workflow task is a 5-tuple:

The formal definition of an enterprise workflow task is a five-tuple:

$$T = (u, p, s_0, \mathcal{A}, G) \tag{3.4}$$

where p denotes organizational business policies, u is the user's natural language request, s_0 is

the initial database state, $\mathcal{A}$ is the set of executable tools, and G is the ground-truth terminal goal criteria.

3.5 Operational Evaluation Metrics and Rubric

To establish an objective, repeatable evaluation standard, execution rollouts are evaluated using a strict multi-component scoring rubric that is directly based on benchmark ground truth. Task evaluation generates several operational measures, including continuous scoring, binary objective attainment, clean execution dependability, and policy constraint adherence.

The continuous goal satisfaction score, denoted by $S_{\text{goal}} \in [0.0, 1.0]$, is a weighted combination of communicated factual accuracy and database action execution:

$$S_{\text{goal}} = 0.6 \times S_{\text{info}} + 0.4 \times S_{\text{act}} \tag{3.5}$$

where S_{info} calculates the percentage of required ground-truth entities and status values that are correctly given in the final customer response, and S_{act} displays the percentage of required state changes and database queries that are correctly executed. The task goal satisfaction accuracy score is represented as a percentage ($S_{\text{goal}} \times 100\% \in [0.0\%, 100.0\%]$). The per-depth and per-tier accuracy in Table 5.2 and Table 5.3 show the average accuracy of continuous goal satisfaction over all the tasks in each category. So they show fractional percentages instead of discrete jumps. The binary task success is recorded by the boolean flag success. It is rewarded if the score of goal satisfaction is greater than a threshold of 70% ($S_{\text{goal}} \geq 0.70$). At terminal recursion level ($r = 3$), 62 Text ReAct tasks (54.4%) and 64 Latent ReAct tasks (56.1%) out of 114 tasks achieve universal binary success. Clean success tasks are defined as the rigorous subset of tasks at terminal depth ($r = 3$) that achieve binary success ($S_{\text{goal}} \geq 0.70$) without causing any tool execution exceptions, malformed argument structures, or policy violations during their rollout (30 tasks for Text ReAct versus 33 tasks for Latent ReAct). The arithmetic average of continuous goal fulfillment accuracy over all four recursion depths ($r = 0 \ldots 3$) is represented by the mean task completion accuracy across recursion depths (50.3% vs. 50.9%). Computational inference expense in TeraFLOPs (TFLOPs) is analytically estimated using standard scaling formulae for Transformer autoregression [7]:

$$\text{FLOPs}_{\text{task}} = \frac{2 \times P \times (N_{\text{prompt}} + N_{\text{gen}}) + \text{FLOPs}_{\text{adapters}}}{10^{12}} \tag{3.6}$$

where $P = 4.4 \times 10^9$ represents the total model parameters (all parameters participate in each dense forward pass, as Qwen3.5-4B is a dense non-MoE architecture), N_{prompt} is the total prompt tokens ingested across all roles (Planner, Tool-Caller, Verifier) and passes ($r \in \{0, 1, 2, 3\}$), and N_{gen} is the total generated tokens. The neural adapters contribute about 2.6×10^7 FLOPs(R_{in}) and 6.5×10^7 FLOPs(R_{out}) FLOPs per pass ($< 0.001\%$ of backbone forward compute). The size of gated DeltaNet linear attention layers is $O(N \cdot d_h^2)$ without quadratic memory matrices in the hybrid architecture of Qwen3.5-4B [5]; for the subset of

full self-attention layers inside our evaluated sequence budget ($N_{\text{turn}} \leq 2048$ tokens), dense projections ($2P$ FLOPs/token) account for more than 97% of theoretical computation. We thus ignore the minor quadratic attention term as a simple analytical approximation, recognizing this constraint while pointing out that precise runtime procedures rely on hardware caching. According to Table 5.1, cumulative analytical estimations for all 456 tasks per architecture add up to 13,076 TFLOPs for Text ReAct and 11,038 TFLOPs for Latent ReAct. precisely matching the total depth expenditures per run plotted in Figure 5.2.

Using the fixed benchmark denominator of 114 tasks (28 tasks or 24.6%), benchmark error recovery yield is defined across the entire benchmark cohort ($N = 114$ tasks) as the percentage of all tasks that showed an initial-pass ($r = 0$) execution failure or malformed invocation but successfully achieved binary goal completion (is_success = True) upon subsequent recursive deliberation ($r \in \{1, 2, 3\}$; 24.6% for Text ReAct as opposed to 33 assignments or 28.9% for Latent ReAct). Policy violations are evaluated deterministically against business rules when an agent modifies immutable items (e.g., changing addresses on dispatched orders, giving excess refunds, or omitting identity verification). Isolated tool selection accuracy is the percentage of API invocations that correspond to valid ground-truth tool schemas.

3.6 Composite Task Complexity Metric and Statistical Testing Methodology

Enterprise client interactions range widely in complexity, from straightforward status inquiries to complicated multi-step transaction disputes. To give an impartial and systematic evaluation, we created a composite difficulty score metric C:

$$C = 1.0 \times C_{\text{tool}} + 2.0 \times C_{\text{write}} + 1.0 \times C_{\text{div}} \tag{3.7}$$

where the distinct tool diversity count is represented by C_{div}, the number of database state changes is represented by C_{write}, and the minimum required tool invocations is represented by C_{tool}.

Using the composite score C, the 114 tasks were split into three balanced tiers of 38 tasks each. The Easy Tier ($C \leq 2$) includes read-only queries with one to two API calls. The Medium Tier ($3 \leq C \leq 5$) requires three to four API requests and one database modification (e.g., updating shipping details following user verification). The Hard Tier ($C \geq 6$) includes five or more API calls, several state changes, relational dependencies, and strict policy validations (such multi-item returns with restocking deductions and stock replacement checks).

In all systems, 912 full trajectories are generated by considering the stated recursion levels $r \in \{0, 1, 2, 3\}$. Formal inferential hypothesis testing is specifically concerned with representation dynamics: step-wise trajectory drift contraction and inter-agent directional alignment across the $N = 114$ task pairings are evaluated using paired, two-sided Wilcoxon signed-rank tests. Non-parametric effect sizes between successful and unsuccessful rollouts are measured

using the Mann-Whitney U test and Cliff's Delta (δ). Family-wise error rates are tightly controlled across the family of $m = 10$ candidate geometric hypotheses using Holm-Bonferroni corrections. All indicated confidence intervals are non-parametric percentile intervals computed over 10,000 bootstrap resamples. However, total benchmark accuracy differences (+0.6 percentage points across depths, +2.0 percentage points at terminal depth) are reported as observed descriptive differences across the benchmark cohort instead of formal statistical significance claims.

Chapter 4

Geometric and Error Analysis of Latent Representations

4.1 Theoretical Formulation of Geometric Representation Metrics

We organize the agent representations as trajectories on a continuous manifold $\mathcal{M} \subset \mathbb{R}^{d_h}$ with hidden dimension $d_h = 2560$ in order to characterize the internal deliberative dynamics of multi-agent execution. The system representation state H_r, which records the unified representational snapshot over recursive rounds $r \in \{0, 1, 2, 3\}$, is examined.

Directional alignment measures how consistently the system's representations are orientated across a series of recursive passes using cosine similarity:

$$\text{CosSim}(H_r, H_{r-1}) = \frac{H_r \cdot H_{r-1}}{\|H_r\|_2 \, \|H_{r-1}\|_2} \quad \text{for } r \in \{1, 2, 3\} \tag{4.1}$$

Representation drift, or D_r, measures the step-wise distance traveled on the representation manifold between subsequent rounds:

$$D_r = 1 - \text{CosSim}(H_r, H_{r-1}) \quad \text{for } r \in \{1, 2, 3\} \tag{4.2}$$

where trajectory shifts are measured in stages by $D_1 = D(H_0 \to H_1)$, $D_2 = D(H_1 \to H_2)$, and $D_3 = D(H_2 \to H_3)$. It is possible to ascertain if multi-agent deliberation converges to a localized representation or continues to wander across state space by tracking D_r during subsequent passes.

We measure the effective dimensionality of the representation manifold across tasks using the Participation Ratio [8, 9]:

$$\text{PR}(H) = \frac{(\sum_{i=1}^{K} \lambda_i)^2}{\sum_{i=1}^{K} \lambda_i^2} \tag{4.3}$$

where λ_i represents the eigenvalues of the centered covariance matrix computed across task representations. Unlike straightforward discrete rank metrics, the Participation Ratio quanti-

fies the continuous effective dimensionality covered by the representation manifold and shows whether recurrent recursion compresses the representational subspace or results in catastrophic dimensional collapse.

We examine the relationship between representation geometry and task resolution by evaluating statistical effect sizes between successful and unsuccessful rollouts using Cliff's Delta (δ):

$$\delta = \frac{\#(\mathcal{S}_{\text{succ}} > \mathcal{S}_{\text{fail}}) - \#(\mathcal{S}_{\text{succ}} < \mathcal{S}_{\text{fail}})}{n_{\text{succ}} \cdot n_{\text{fail}}} \tag{4.4}$$

complemented by non-parametric Mann-Whitney U tests across task trajectories and Holm-Bonferroni family-wise error rate corrections over the whole family of $m = 10$. Five possible geometric predictors for both designs are evaluated through hypothesis testing: final-step drift, trajectory tortuosity, net trajectory drift, cumulative path length, and representation L_2 norm.

4.2 Inferences from Representation Drift and Convergence Dynamics

The trajectory dynamics of text-mediated and latent recursion differ significantly, according to a comprehensive empirical investigation of all 114 retail tasks. During the initial transition from round zero to round one, the system state in Latent ReAct exhibits an instantaneous, decisive movement with a mean cosine distance of zero point zero six five. Crucially, this single initial transition accounts for 64% of the total trajectory displacement during the three-round recursive budget. In the subsequent rounds, the step-wise displacement considerably decreases, exhibiting a near-stationary drift of zero point zero one eight between rounds one and two and zero point zero one nine between rounds two and three. A paired Wilcoxon signed-rank test confirms that this step-wise reduction from the first step to subsequent steps is highly statistically significant ($p = 1.9 \times 10^{-20}$), suggesting that latent recursion rapidly stabilizes within a constrained representational domain.

In contrast, Multi-Round Text ReAct does not exhibit similar stability and maintains continuous, unattenuated representational drift during all recursive runs. Text-mediated system states drift by zero point zero three eight in the first transition, zero point zero three two in the second, and zero point zero three three in the third, accounting for only thirty-seven percent of the total trajectory movement. Paired non-parametric testing reveals significant differences between latent and text drift profiles at each recursion step ($p < 10^{-10}$).

To rule out the chance that this measured contraction is an artifact of cached or un-updated agent states, an independent robustness analysis was performed by removing all sixty-six jobs displaying bit-identical agent representations over successive rounds. On the remaining 48 carefully updated tasks, the latent contraction is completely resilient, shifting from zero point zero six nine to zero point zero two one ($p = 7.1 \times 10^{-15}$). However, during each round ($p = 0.96$), the text baseline remains entirely flat with an invariant drift of zero point zero four zero.

A foundation for comprehending the theoretical underpinnings of this disparity is offered by discrete dynamical systems. Dynamical systems theory states that an iterative mapping $h_{r+1} = R(h_r)$ exhibits fixed-point attraction if it meets a contractive condition in a confined neighborhood surrounding an equilibrium state. Although a parameterized residual Multi-Layer Perceptron ($R(h) = h + W_2\sigma(W_1 h)$) can not logically guaranty contractivity across all possible inputs without explicit Lipschitz norm constraints, our empirical results demonstrate significant convergence-like behavior in practice. After the fundamental task context is included in the initial latent update, subsequent residual perturbations decrease by more than 70%, limiting the trajectory to a stable local attractor basin. Nevertheless, in each cycle of text recursion, autoregressive sampling generates new natural language critique tokens; re-encoding these verbal strings introduces stochastic linguistic variability, which consistently interferes with internal activations and prevents trajectory stabilization.

4.3 Dimensionality Analysis and Subspace Compression

The Participation Ratio for over two hundred random subsets of ninety tasks suggests that Latent ReAct uses a strictly lower-dimensional and more focused representational subspace than Text ReAct. So at round zero the text occupies twenty-two point zero dimensions but the latent representation manifold occupies an average of eighteen point nine effective dimensions. Both designs exhibit continuous representational compression over iterative runs. By round three their effective dimensionality (fifteen point two dimensions for latent, sixteen point eight dimensions for text) has been reduced by about twenty to twenty-three percent.

This interpretation is consistent with the persistent dimensionality gap observed across more than 97% of bootstrap subsamples, where continuous latent deliberation filters away irrelevant lexical variation and constrains multi-agent reasoning to a compact semantic subspace directly relevant to task constraints. Importantly, recurrent latent passing does not suffer from catastrophic dimensional collapse, as it preserves sufficient representational capacity to encode multi-step corporate workflows, as evidenced by the preservation of 15.2 effective dimensions at full recursion depth.

To determine whether individual agent roles preserve distinct functional identities within this compact subspace, we evaluate inter-agent directional alignment using cosine similarity $\mathrm{CosSim}(A_i, A_j)$ and Euclidean distance $D_{ij} = \|H_i - H_j\|_2$ across the Planner (A_1), Tool-Caller (A_2), and Verifier (A_3). With an average cosine similarity of $\mathrm{CosSim}(A_1, A_2) = 0.087$ between Planner and Tool-Caller representations and $\mathrm{CosSim}(A_2, A_3) = 0.049$ between Tool-Caller and Verifier, the average inter-agent Euclidean distance is $D_{ij} \approx 51.5$. Agent activations in Latent ReAct preserve functional specialization that is almost orthogonal. A two-sample Kolmogorov-Smirnov test reveals a substantial difference in the inter-agent cosine similarity distribution in Latent ReAct and Text ReAct ($p < 10^{-15}$). With an average cosine similarity $\mathrm{CosSim} > 0.998$ and a compressed Euclidean distance of $D_{ij} \approx 10.6$, internal activations in Text ReAct exhibit significant overlap. This is due to the fact that verbose dialog history

is appended to a shared prompt to establish each agent's context. This empirical difference supports the idea that continuous tensor routing through the OuterLink adapters maintains functional role specialization rather than collapsing into representational redundancy.

4.4 Failure Diagnostics and the Null Outcome Relationship

An important scientific challenge is whether global geometric measurements can directly distinguish between successful task completions and execution failures. In order to determine Cliff's Delta effect sizes and bootstrap confidence intervals across five candidate geometric features, we compared thirty-three clean-success rollouts against eighty-one non-clean or failed rollouts in Latent ReAct and thirty clean-success rollouts against eighty-four non-clean or failed rollouts in Text ReAct (using the same evaluation partitioning tasks under general binary success: sixty-four vs. fifty in Latent, sixty-two vs. fifty-two in Text). The empirical study establishes a clear and accurate negative consequence. All candidate features, such as net trajectory drift ($\delta = 0.093, p = 0.438$), cumulative path length ($\delta = 0.140, p = 0.245$), trajectory tortuosity ($\delta = 0.036, p = 0.769$), final-step drift ($\delta = 0.167, p = 0.164$), and final representation norm ($\delta = -0.058, p = 0.630$), have 95% bootstrap confidence intervals that include zero. Across the family of statistical hypotheses, no geometric metric achieves significance following Holm-Bonferroni adjustment (minimum adjusted $p = 0.54$). Similarly, task complexity tiers do not alter representation drift (Kruskal-Wallis $p = 0.73$), and cross-validated linear probes trained on global representations are unable to decode task difficulty above chance levels (balanced accuracy of zero point two nine to zero point three eight against chance baseline zero point three three).

This strict null finding offers an important technical insight: the macroscopic, pooling system representations (H_r) do not geometrically encode downstream job fulfillment linearly. Because H_r is an average across many functional roles, averaging smooths over local, agent-specific signals that control tool execution integrity. Interestingly, an exploratory linear probe on isolated Tool-Caller representations showed a marginal predictive signal for task outcome (Area Under the Curve of 0.64, permutation $p = 0.05$) (H_2). We explicitly state that independent validation in separate benchmark cohorts is required before any firm conclusions can be drawn and we strongly maintain this as an exploratory finding. While macroscopic deliberation geometry captures overall trajectory convergence and subspace compression, not a proven predictor of task success, this first finding suggests localized, tool-specific representations may warrant further study as possible correlates of execution reliability.

Chapter 5

Results and Discussions

5.1 Quantitative Benchmark Performance Evaluation

This chapter uses empirical data and in-depth discussions from 912 benchmark evaluations on the stateful τ^2-Bench Retail domain to compare Multi-Round Text ReAct to Recursive Latent ReAct across 114 jobs assessed at recursion levels $r \in \{0, 1, 2, 3\}$.

Table 5.1 summarizes the benchmark performance for each of the 912 execution runs. We distinguish clearly between: (i) the continuous goal satisfaction accuracy ($S_{\text{goal}} \times 100\%$) averaged over all recursion depths ($r \in \{0, 1, 2, 3\}$)—reported as an observable descriptive difference of 50.9% for Latent ReAct compared to 50.3% for Text ReAct (+0.6 pp), increasing to 56.4% as opposed to 54.4% at terminal depth ($r = 3$, a gain of 2.0 pp); (ii) Out of 114 tasks, 64 tasks (56.1%) for Latent ReAct and 62 tasks (54.4%) for Text ReAct achieved the general binary task success rate ($S_{\text{goal}} \geq 0.70$) at terminal depth; and (iii) 33 tasks for Latent ReAct and 30 tasks for Text ReAct achieved clean terminal success ($S_{\text{goal}} \geq 0.70$ without tool exceptions or policy violations). Latent ReAct performs better than Text ReAct at the task level, with an error recovery yield of 33 initially failing tasks. (28.9% across all 114 benchmark tasks) over the course of future recursive passes, compared to 28 tasks (24.6%). Latent ReAct improves goal accuracy on hard processes by 4.9 percentage points (50.8% versus 45.9%) and decreases enterprise policy violations by 23.1% (10 versus 13 jobs). Latent ReAct simultaneously achieves a 25.2% decrease in generated tokens, a 15.6% reduction in total inference computation, and an observed 26.1% reduction in mean end-to-end latency (8.35 seconds against 11.30 seconds) on the examined benchmark environment.

Table 5.2 displays the sequential evolution of task goal satisfaction accuracy, latency, tool precision, and token consumption spanning recursion levels zero to three. Both architectures show how recursive deliberation works by raising target accuracy from about 45% at round zero to over 54% at round three. On the other hand, Latent ReAct scales with a far higher computational efficiency, finishing round three in 13.83 seconds as opposed to Text ReAct's 18.56 seconds.

Table 5.3 looks at the terminal performance and latency scaling achieved at full recursion depth ($r = 3$) across job complexity tiers. The cohort's single-run execution latencies at $r = 3$

Table 5.1: Overall Benchmark Performance Summary for 912 Execution Runs on τ^2-Bench Retail Domain.

Performance Dimension / Metric	Text ReAct	Latent ReAct	Relative Advantage
Mean Goal Accuracy ($r = 0 \dots 3$)	50.3%	**50.9%**	**+0.6 pp (Latent)**
Terminal Goal Accuracy ($r = 3$)	54.4%	**56.4%**	**+2.0 pp (Latent)**
Terminal Binary Success ($S_{\text{goal}} \geq 0.70$)	54.4% (62 tasks)	**56.1% (64 tasks)**	**+1.7 pp (Latent)**
Hard Workflow Goal Accuracy	45.9%	**50.8%**	**+4.9 pp (Latent)**
Clean Success Tasks (Terminal $r = 3$, out of 114)	30 tasks (26.3%)	**33 tasks (28.9%)**	**+10.0% (Latent)**
Benchmark Error Recovery Yield (out of 114)	24.6% (28 tasks)	**28.9% (33 tasks)**	**+17.9% (Latent)**
Policy Constraint Violations	13 tasks	**10 tasks**	**23.1% Reduction (Latent)**
Mean End-to-End Latency	11.30 s	**8.35 s**	**26.1% Observed Reduction (Latent)**
Total Output Generated Tokens	166,779 tokens	**124,805 tokens**	**25.2% Savings (Latent)**
Total Inference Computation	13,076 TFLOPs	**11,038 TFLOPs**	**15.6% Savings (Latent)**
Isolated Tool Selection Accuracy	**57.5%**	54.2%	-3.3 pp (Latent - Text)

Table 5.2: Benchmark Evaluation Breakdown by Recursion Depth ($r = 0, 1, 2, 3$), with a mean continuous accuracy of goal satisfaction ($S_{\text{goal}} \times 100\%$).

	Text ReAct (Baseline)				Recursive Latent ReAct (Ours)			
Depth	**Goal Acc (%)**	**Lat (s)**	**Tool (%)**	**Tokens**	**Goal Acc (%)**	**Lat (s)**	**Tool (%)**	**Tokens**
$r = 0$	45.7	4.36	58.8	16,891	45.3	**3.05**	56.1	**11,719**
$r = 1$	48.2	8.82	57.9	33,442	**48.5**	**6.52**	52.2	**25,077**
$r = 2$	52.8	13.45	57.3	49,412	**53.3**	**9.98**	55.0	**37,387**
$r = 3$	54.4	18.56	56.1	67,034	**56.4**	**13.83**	53.5	**50,622**

include the total inference time over four full deliberation runs, with an average of 18.56 s for Text ReAct and 13.83 s for Latent ReAct (see Table 5.2). On the other hand, the grand mean latencies shown in Table 5.1 (11.30 s for Text ReAct vs. 8.35 s for Latent ReAct) indicate the average operating cost for all 912 runs over all recursion depths ($r \in \{0, 1, 2, 3\}$).10.18 s vs. 7.71 s on Easy, 11.71 s vs. 8.36 s on Medium, and 12.00 s vs. 8.97 s on Hard are the tier-specific averages for all depths. On hard processes involving multi-entity coordination, replacement checks, and restocking policy deductions, Latent ReAct performs noticeably better than Text ReAct (50.8% vs. 45).Text ReAct maintains a little advantage over Medium single-mutation processes, with 9% accuracy and 24.5% faster execution at $r = 3$.

Table 5.3: Task Complexity Tier Performance and Execution delay at Terminal Depth $r = 3$ (single-run execution delay and reported continuous goal fulfillment accuracy $S_{\text{goal}} \times 100\%$).

	Text ReAct		Recursive Latent ReAct	
Complexity Tier	**Terminal Goal Acc (%)**	**Terminal Latency (s)**	**Terminal Goal Acc (%)**	**Terminal Latency (s)**
Easy Tier (38 Tasks)	53.8%	16.67 s	**57.5%**	**12.89 s**
Medium Tier (38 Tasks)	**63.7%**	19.54 s	60.9%	**13.93 s**
Hard Tier (38 Tasks)	45.9%	19.46 s	**50.8%**	**14.68 s**

5.2 Behavioral Accuracy and Computational Scaling

Figure 5.1 displays the evolution of overall task completion accuracy across recursion rounds $r \in \{0, 1, 2, 3\}$. Both architectures exhibit monotonic performance gains as recursion depth rises from $r = 0$ to $r = 3$. In contrast, Latent ReAct exhibits a faster accuracy trend from 45.3

percent at round zero to 56.4 percent at round three, outperforming Text ReAct by an observed two percentage points at full recursion depth. In complex workflows, the accumulation of verbose textual self-critique within the prompt context may lead to model fixation on early errors, potentially creating needless token history. The idea that Latent ReAct maintains relational problem representations in continuous space, avoiding cumulative textual history bloat and permitting multi-step error recovery, is supported by our recovery yield measurements. Figure 5.2 displays the scaling of mean computational inference floating-point operations per task run throughout recursive depths $r \in \{0, 1, 2, 3\}$. It is important to distinguish between the per-run operational scale plotted in Figure 5.2 and the benchmark-wide totals shown in Table 5.1. The total cumulative sum of all floating-point operations throughout all 456 execution runs on the 114 benchmark tasks is the stated computational volume (13,076 TFLOPs for Text ReAct versus 11,038 TFLOPs for Latent ReAct) in Table 5.1. The presented curves in Figure 5.2 illustrate the average computational cost per individual task execution at each discrete recursion depth. The per-run compute scales from seven point eight nine TFLOPs for Text ReAct and seven point zero seven TFLOPs for Latent ReAct at round zero ($r = 0$) to thirty-four point nine four versus twenty-nine point three six TFLOPs at round two ($r = 2$) and fifty-two point eight four TFLOPs for Text ReAct versus forty-three point nine two TFLOPs for Latent ReAct at round three ($r = 3$). This results in a sixteen percent compute reduction per task at full recursion depth. The autoregressive creation of explicit natural language tokens required by Text ReAct's intermediate stages results in a significant increase in computation per run. Latent ReAct uses matrix multiplications in lightweight neural adapters for all intermediate deliberation stages, avoiding token generation overhead and achieving a 15.6% reduction in cumulative benchmark computation.

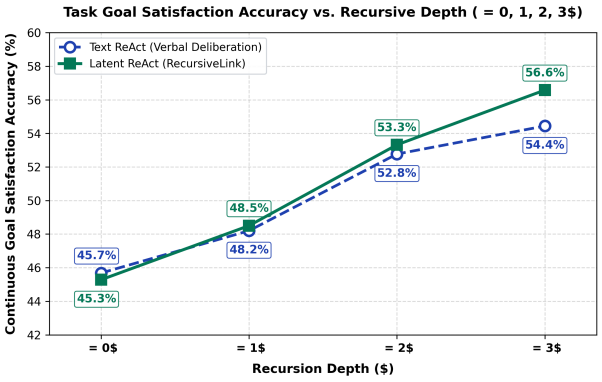


Figure 5.1: Total task completion accuracy as a function of recursive depth ($r = 0, 1, 2, 3$) for 114 corporate jobs comparing Multi-Round Text ReAct to Recursive Latent ReAct.

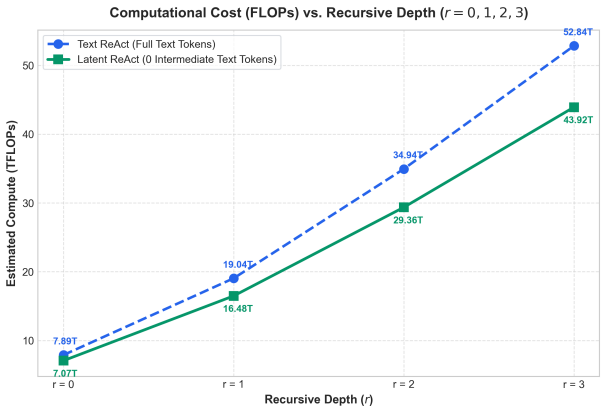


Figure 5.2: Table 5.1 shows the average computational inference expenditure per task run (TFLOPs) over recursion depths ($r \in \{0, 1, 2, 3\}$), comparing single-run scaling curves with benchmark-wide cumulative totals.

5.3 Geometric State Convergence and Dimensionality Results

Figure 5.3 displays the empirical step-wise state drift across recursion passes along with 95% bootstrap confidence intervals. The visual trajectory experimentally validates finding 1 from our geometric analysis: Latent ReAct concentrates 64% of its total displacement in the initial transition ($r = 0 \rightarrow 1$), falling sharply from 0.065 to 0.018 before stabilizing. Text ReAct, on the other hand, exhibits a drift that is approximately constant, averaging 0.034 every step (0.038, 0.032, and 0.033 across consecutive passes). A paired Wilcoxon signed-rank test over all 114 tasks shows that this stepwise decline from first to subsequent transitions is highly significant ($p = 1.9 \times 10^{-20}$). Even after removing tasks with duplicate states, the latent contraction remains strong ($p = 7.1 \times 10^{-15}$), whereas the text drift is flat ($p = 0.96$). Technically, we do not provide a guaranty of fixed-point contractivity with the unconstrained residual adapters, but we do observe empirically a substantial stabilization of the trajectory toward a compact manifold, similar to convergence. We find that trajectory stabilization is an empirical geometric pattern, rather than a direct, verified cause of task completion.

Figure 5.4: Participation Ratio measured over two hundred bootstrap subsets for the effective dimensionality of the representation manifold. In each round latent ReAct has a smaller, lower dimensional subspace than Text ReAct (18.9 $\rightarrow$ 15.2 dimensions for latent vs. 22.0 $\rightarrow$ 16.8 dimensions for text). Both architectures achieve 20–23% compression over recursive cycles. This is consistent with the idea that latent communication, by removing unnecessary linguistic variability, preserves a compact, non-collapsing manifold that is suitable for enterprise thinking.

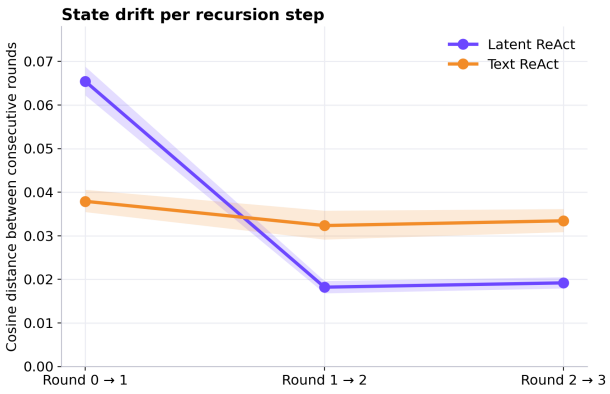


Figure 5.3: 95% bootstrap confidence intervals for stepwise representation drift over recursive passes.

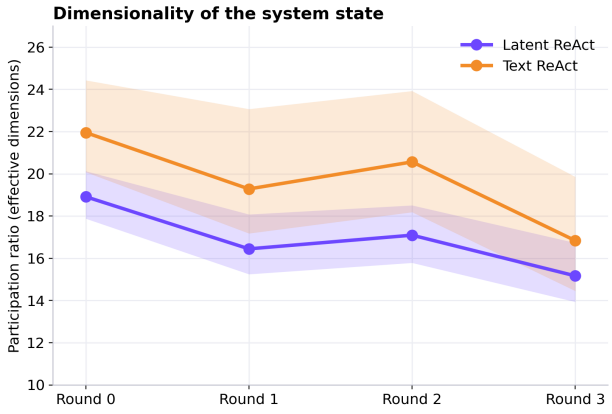


Figure 5.4: Over 200 bootstrap subsets were used to evaluate the participation ratio and effective dimensionality across recursive passes.

The effect sizes (Cliff's δ with 95% bootstrap confidence intervals) for each of the five possible trajectory metrics that compare clean successes to failures are shown in Figure 5.5. Across the family of statistical hypotheses, all intervals embrace zero after Holm-Bonferroni

correction (minimum adjusted $p = 0.54$), empirically proving that downstream transaction success is not only determined by macroscopic representation geometry.

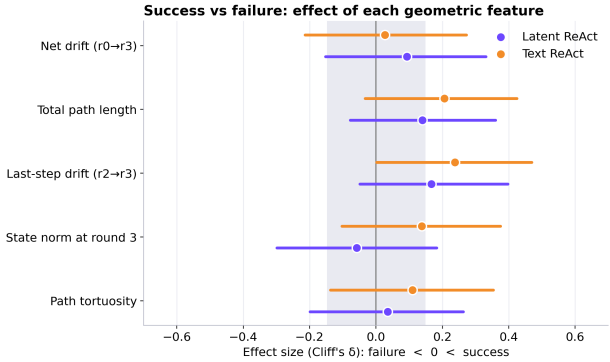


Figure 5.5: A null connection with global representation geometry was confirmed by comparing clean successes against failures using effect sizes (Cliff's δ with 95% bootstrap confidence intervals) across five possible trajectory features.

5.4 Discussion of the Tool Selection Trade-Off

In single-step isolated tool selection, Text ReAct's observed descriptive accuracy was 57.5%, whereas Latent ReAct's was 54.2%. This shows a 3.3 percentage point negative difference (Latent minus Text; bootstrap 95 percent confidence interval of $[-5.1\%, -1.5\%]$). This observed variance reflects the superficial lexical advantage of Text ReAct, where clear API names and schema definitions are instantly accessible in the prompt strings. Text ReAct's lower ultimate accuracy of 45 could be explained in one way.9% on Hard tasks is that unrestrained verbal reflection finds it difficult to correct compounding relational faults on complicated workflows when there is no ongoing trajectory stabilization. On the other hand, continual latent feedback seems to preserve relational consistency during deliberation passes, enabling Latent ReAct to achieve higher terminal accuracy and resolve 10% more corporate jobs cleanly.

Chapter 6

Conclusion and Scope for Further Research

In order to examine the computational effectiveness, latency, and dependability of autonomous language agents in stateful corporate workflows, this thesis presented and empirically assessed a Recursive Multi-Agent System that communicates through continuous latent representations. Latent ReAct increased Hard-tier goal accuracy to 50.8% across 912 benchmark evaluations on the stateful τ^2-Bench Retail domain, while Text ReAct's improvement was 45.9%. Additionally, it produced 25.2% less tokens, decreased mean latency by 26.1%, increased clean terminal successes by 10% (33 versus 30), and decreased policy violations from 13 to 10. Trajectory stabilization was empirically demonstrated by geometric analysis: Latent ReAct concentrated about 64% of its total trajectory displacement in the initial recursive transition, with a statistically significant decrease in subsequent drift ($p = 1.9 \times 10^{-20}$). Additionally, the duplicate-filtered analysis revealed a contraction of almost 70% ($p = 7.1 \times 10^{-15}$). Nevertheless, without suitable Lipschitz restrictions, the residual adapters do not mathematically provide fixed-point contractivity. At terminal depth, the latent representations maintained 15.2 effective dimensions, as opposed to 18.9 at initial depth. After multiple-hypothesis correction, statistical analysis revealed no significant correlation between the tested global geometric measures and task accomplishment, indicating that global, role-averaged representations by themselves are not sufficient to predict execution results. The evaluation encompassed 114 tasks in a single retail domain, and the mean goal-accuracy gain was small (+0.6 percentage points averaged across recursion levels and +2.0 percentage points at terminal depth). These results should be evaluated in the context of the study's constraints.and Latent ReAct's isolated tool-selection accuracy was worse than Text ReAct's (54.2% versus 57.5%; bootstrap 95% CI for the difference: $[-5.1\%, -1.5\%]$). Future research should look into adaptive early-stopping techniques driven by representation drift, hybrid lexical-latent training, and evaluation across more enterprise areas.

Appendix A

Benchmark Environment and Tool Schema Specification

A.1 Retail Domain API Tool Specifications

Table A.1 contains complete technical specifications and functional definitions for the thirteen executable Application Programming Interface tools available in the τ^2-Bench Retail enterprise environment.

Table A.1: Executable application programming interface tool specs.

Tool Name	Arguments	Operational Function
`search_user`	`user_id, email, phone`	locates a client profile in the database and verifies their identity.
`get_user_details`	`user_id`	Acquires client account history, shipment addresses, and payment methods.
`get_order_details`	`order_id`	Obtains itemized line items, tracking and payment status, dates, and prices.
`search_orders`	`user_id, status`	Queries all of a specific customer's prior and current orders.
`calculate_refund`	`order_id, item_ids`	Determines the net refundable balance following policy verification and restocking expenses.
`process_item_return`	`order_id, item_ids, reason`	Generates labels, modifies the state, and logs an item return in the database.
`modify_pending_order`	`order_id, modifications`	Pending order attributes are modified before warehouse fulfillment.
`cancel_order`	`order_id, reason`	Cancels an unfilled order and initiates an automated payment reversal.

Continued on next page

Tool Name	Arguments	Operational Function
update_shipping_address	user_id, address_dict	Updates the customer's primary or order-specific shipping address records.
check_product_inventory	product_ids	Verifies the quantity of replacement goods that the warehouse has on hand.
create_exchange_order	user_id, product_ids	Creates an exchange order for defective or replacement products.
apply_discount_code	order_id, promo_code	Checks promotional discount coupons for compliance with order specifications.
escalate_to_human	user_id, ticket_reason	Notifies human support personnel about edge cases that are unresolvable.

A.2 Relational Database Schema

The underlying SQLite relational database consists of five basic entity tables that maintain state during multi-turn customer transactions. Customer identifiers, contact information, primary shipping addresses, and loyalty membership tiers are kept up to date in the `users` table. Transaction identifiers, customer foreign keys, placement timestamps, fulfillment status, and order totals are all stored in the `orders` table. Product purchases, quantities, unit pricing, return flags, and policy return eligibility are all itemized in the `order_items` table. Warehouse stock levels, product classifications, catalog inventories, and restocking restrictions are all monitored via the `products` database. Lastly, the `returns` table keeps track of inspection statuses, submission dates, return item justifications, and refund authorizations.

References

[1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 8634–8652, 2023.

[2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[3] X. Yang, J. Zou, R. Pan, R. Qiu, P. Lu, S. Diao, J. Jiang, H. Tong, T. Zhang, M. J. Buehler, J. He, and J. Zou, "RecursiveMAS: Recursive multi-agent systems," *arXiv preprint arXiv:2604.25917*, 2026.

[4] R.-J. Zhu, Z. Wang, K. Hua, T. Zhang, Z. Li, H. Que, B. Wei, Z. Wen, F. Yin, and H. Xing, "Scaling latent reasoning via looped language models," *arXiv preprint arXiv:2510.25741*, 2025.

[5] Qwen Team, "Qwen3.5-4B: Next-generation multimodal foundation model." `https://huggingface.co/Qwen/Qwen3.5-4B`, 2026. Alibaba Cloud.

[6] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-Bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[8] B. Sorscher, R. Geirhos, S. Shekhar, S. Ganguli, and A. S. Morcos, "Beyond neural scaling laws: Beating power law scaling via data pruning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 19565–19577, 2022.

[9] O. Roy and M. Vetterli, "The effective rank: A measure of effective dimensionality," in *15th European Signal Processing Conference (EUSIPCO)*, pp. 606–610, 2007.